

**МИНОБРНАУКИ РОССИИ**  
**САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ**  
**ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ**  
**«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)**  
**Кафедра МО ЭВМ**

**ОТЧЕТ**  
**по лабораторной работе №2**  
**по дисциплине «Нейронные сети»**  
**Тема: Бинарная классификация отраженных сигналов радара**

Студент гр. 1306

Шаврин А.П.

Преподаватель

Жангиров Т.Р.

Санкт-Петербург

2026

## **Цель работы.**

Реализовать классификацию между камнями (R) и металлическими цилиндрами (M) на основе данных об отражении сигналов радара от поверхностей. 60 входных значений показывают силу отражаемого сигнала под определенным углом. Входные данные нормализованы и находятся в промежутке от 0 до 1

## **Задание.**

- Ознакомиться с задачей бинарной классификации
- Загрузить данные
- Создать модель ИНС в tf.Keras
- Настроить параметры обучения
- Обучить и оценить модель
- Изменить модель и провести сравнение. Объяснить результаты

## **Выполнение работы.**

В качестве набора данных использовался классический датасет Sonar, содержащий 208 образцов отраженных сигналов сонара. Каждый образец представлен 60 признаками (амплитудами сигналов на разных частотах) и меткой класса: "R" (камень) или "M" (металлический цилиндр). Данные были успешно загружены из файла sonar.csv (рисунок 1).

Предобработка данных включала следующие шаги:

- Разделение признаков и целевой переменной: матрица признаков X (первые 60 столбцов) и вектор меток y (последний столбец).
- Кодирование меток: строковые метки ('R' и 'M') были преобразованы в числовой формат (0 и 1) с помощью LabelEncoder. Это необходимо для использования функции потерь `binary_crossentropy`.
- Разделение на обучающую и тестовую выборки: данные были разделены в соотношении 80/20 с сохранением пропорций классов (стратификация). Для воспроизводимости результатов использован `random_state=42`. 20% от

обучающей выборки использовалось как валидационная выборка внутри процесса обучения.

## Загрузка данных

```
[9]
✓ 0s
dataframe = pd.read_csv("sonar.csv", header=None)
print(dataframe.head())
```

|   | 0      | 1      | 2      | 3      | 4      | 5      | 6      | 7      | 8      | \ |
|---|--------|--------|--------|--------|--------|--------|--------|--------|--------|---|
| 0 | 0.0200 | 0.0371 | 0.0428 | 0.0207 | 0.0954 | 0.0986 | 0.1539 | 0.1601 | 0.3109 |   |
| 1 | 0.0453 | 0.0523 | 0.0843 | 0.0689 | 0.1183 | 0.2583 | 0.2156 | 0.3481 | 0.3337 |   |
| 2 | 0.0262 | 0.0582 | 0.1099 | 0.1083 | 0.0974 | 0.2280 | 0.2431 | 0.3771 | 0.5598 |   |
| 3 | 0.0100 | 0.0171 | 0.0623 | 0.0205 | 0.0205 | 0.0368 | 0.1098 | 0.1276 | 0.0598 |   |
| 4 | 0.0762 | 0.0666 | 0.0481 | 0.0394 | 0.0590 | 0.0649 | 0.1209 | 0.2467 | 0.3564 |   |

|   | 9      | ... | 51     | 52     | 53     | 54     | 55     | 56     | 57     | \ |
|---|--------|-----|--------|--------|--------|--------|--------|--------|--------|---|
| 0 | 0.2111 | ... | 0.0027 | 0.0065 | 0.0159 | 0.0072 | 0.0167 | 0.0180 | 0.0084 |   |
| 1 | 0.2872 | ... | 0.0084 | 0.0089 | 0.0048 | 0.0094 | 0.0191 | 0.0140 | 0.0049 |   |
| 2 | 0.6194 | ... | 0.0232 | 0.0166 | 0.0095 | 0.0180 | 0.0244 | 0.0316 | 0.0164 |   |
| 3 | 0.1264 | ... | 0.0121 | 0.0036 | 0.0150 | 0.0085 | 0.0073 | 0.0050 | 0.0044 |   |
| 4 | 0.4459 | ... | 0.0031 | 0.0054 | 0.0105 | 0.0110 | 0.0015 | 0.0072 | 0.0048 |   |

|   | 58     | 59     | 60 |
|---|--------|--------|----|
| 0 | 0.0090 | 0.0032 | R  |
| 1 | 0.0052 | 0.0044 | R  |
| 2 | 0.0095 | 0.0078 | R  |
| 3 | 0.0040 | 0.0117 | R  |
| 4 | 0.0107 | 0.0094 | R  |

[5 rows x 61 columns]

## Предобработка данных

```
[11]
✓ 0s
X = dataset[:, 0:60].astype(float)
Y = dataset[:, 60]

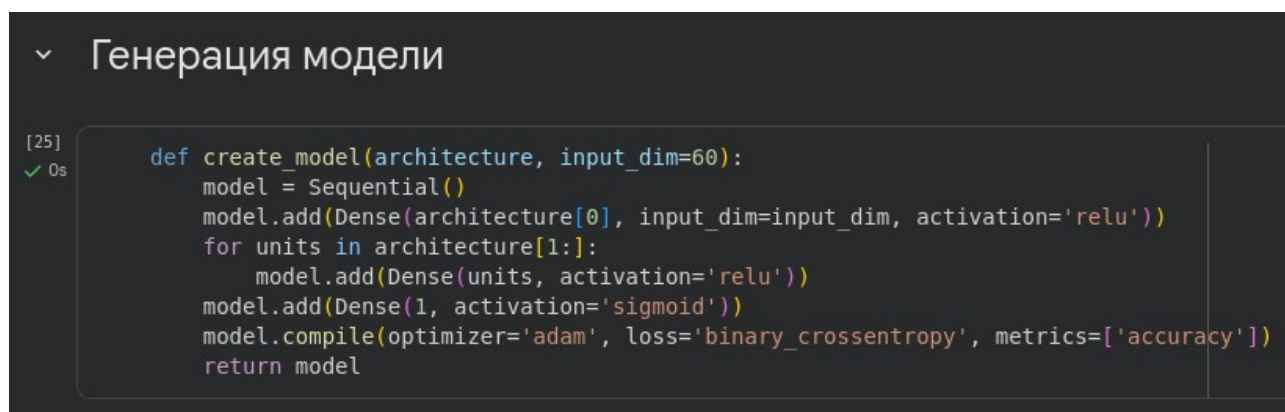
encoder = LabelEncoder()
encoder.fit(Y)
encoded_Y = encoder.transform(Y)

X_train, X_test, y_train, y_test = train_test_split(
    X, encoded_Y, test_size=0.2, random_state=42, stratify=encoded_Y
)
```

Рисунок 1. Чтение и предобработка данных.

Для проведения экспериментов была написана функция `create_model()` (рисунок 2), позволяющая задавать архитектуру сети через список `architecture`, где каждый элемент соответствует количеству нейронов в скрытом слое. Модель имеет входной слой, соответствующее количество скрытых слоёв с активацией ReLU и выходной слой с одним нейроном и функцией активации

sigmoid для бинарной классификации. Компиляция выполнялась с оптимизатором Adam и функцией потерь binary\_crossentropy.



The screenshot shows a Jupyter Notebook interface with a dark theme. At the top, there is a section header 'Генерация модели' with a dropdown arrow. Below it, a code cell is displayed with the following Python code:

```
[25]
✓ 0s
def create_model(architecture, input_dim=60):
    model = Sequential()
    model.add(Dense(architecture[0], input_dim=input_dim, activation='relu'))
    for units in architecture[1:]:
        model.add(Dense(units, activation='relu'))
    model.add(Dense(1, activation='sigmoid'))
    model.compile(optimizer='adam', loss='binary_crossentropy', metrics=['accuracy'])
    return model
```

Рисунок 2. Функция создания модели.

### 1. Сравнение архитектур нейронной сети

В этом эксперименте было обучено 8 моделей с различной архитектурой – от простой однослойной до двухслойных с разным числом нейронов. Список исследованных архитектур:

- [15] – один скрытый слой с 15 нейронами.
- [30] – один скрытый слой с 30 нейронами.
- [60] – один скрытый слой с 60 нейронами.
- [120] – один скрытый слой с 120 нейронами.
- [60, 30] – два скрытых слоя (60 и 30 нейронов).
- [60, 15] – два скрытых слоя (60 и 15 нейронов).
- [30, 15] – два скрытых слоя (30 и 15 нейронов).
- [16, 8] – два скрытых слоя (16 и 8 нейронов).

Обучение каждой модели проводилось в течение 100 эпох с размером батча, равным 10. На рисунках 3 и 4 представлены графики точности и потерь для каждой из архитектур, а в таблице 1 значения точности для каждой архитектуры на валидационных и тестовых данных.

### Графики точности для всех архитектур

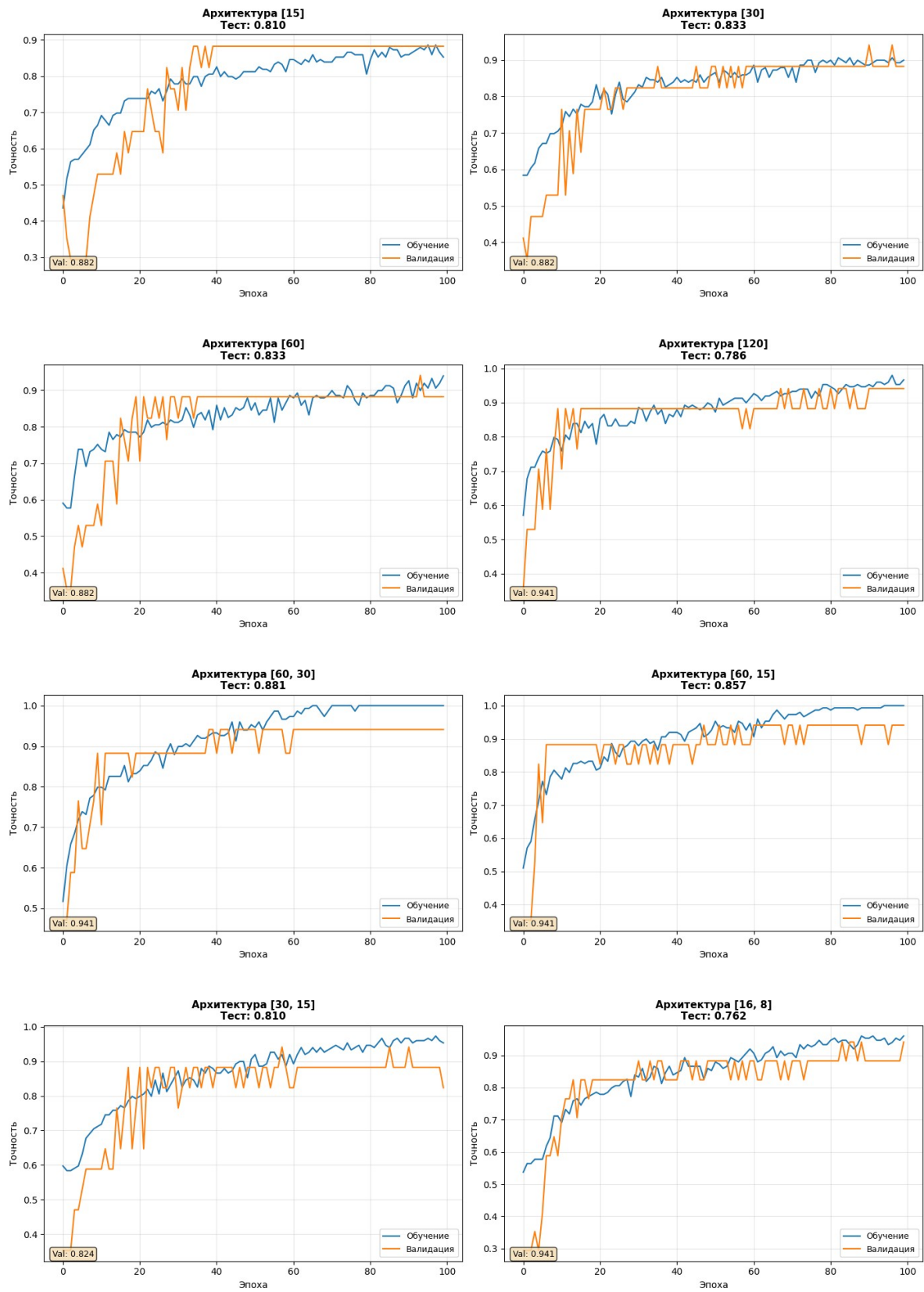


Рисунок 3. Графики точности для всех архитектур.

# Графики потерь для всех архитектур

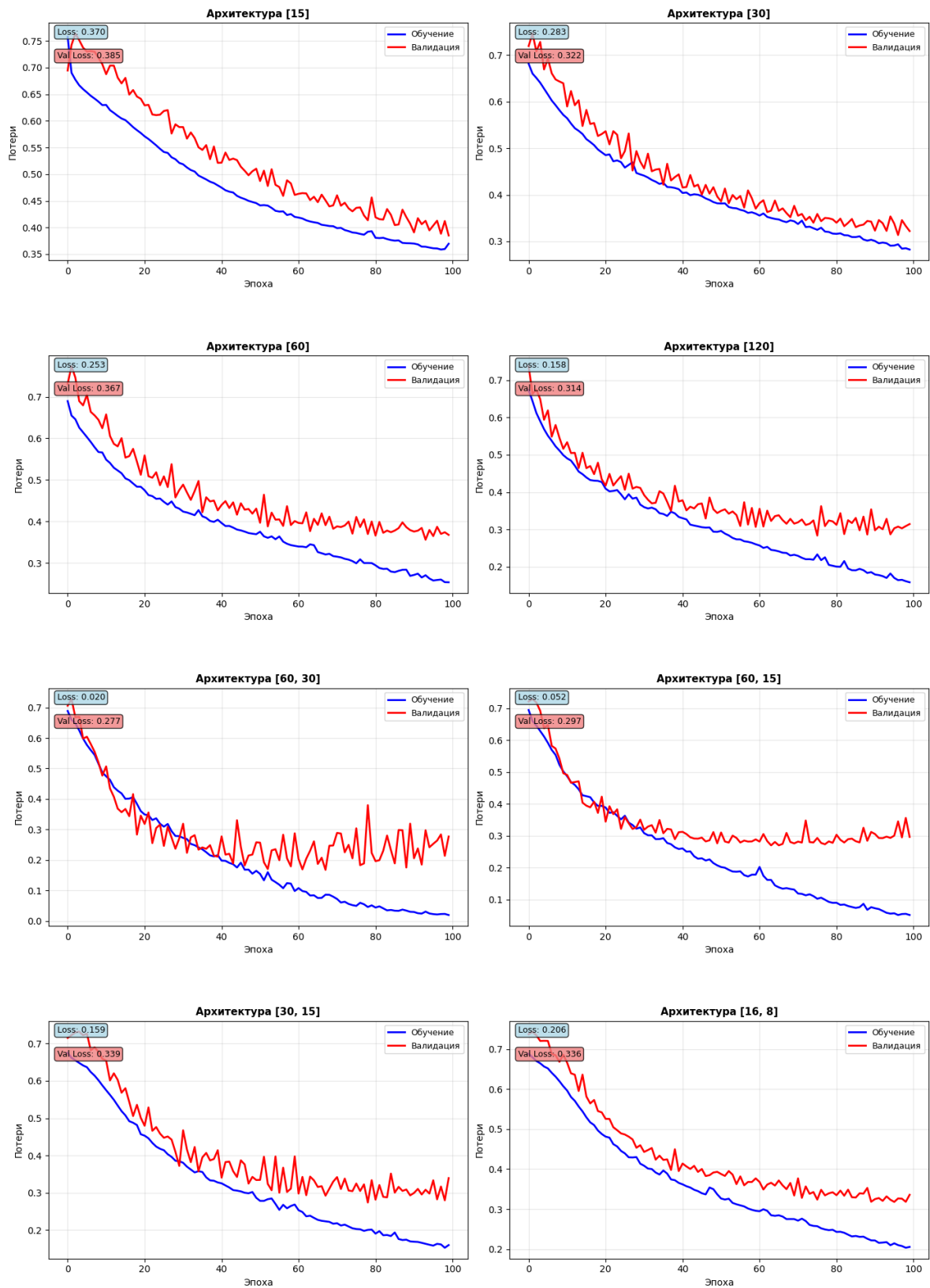


Рисунок 4. Графики потерь для всех архитектур.

Таблица 1. Результаты обучения различных архитектур.

| Архитектура | Точность на валидации | Точность на тесте |
|-------------|-----------------------|-------------------|
| [15]        | 0.8824                | 0.8095            |
| [30]        | 0.8824                | 0.8333            |
| [60]        | 0.8824                | 0.8333            |
| [120]       | 0.9412                | 0.7857            |
| [60, 30]    | 0.9412                | <b>0.8810</b>     |
| [60, 15]    | 0.9412                | 0.8571            |
| [30, 15]    | 0.8235                | 0.8095            |
| [16, 8]     | 0.9412                | 0.7619            |

Анализируя полученные графики и таблицу, можно сделать следующие наблюдения:

- Однослойные архитектуры [15], [30], [60] показывают близкие результаты на тесте (0.8095–0.8333). Увеличение числа нейронов с 15 до 60 дает небольшой прирост точности. Графики обучения демонстрируют плавное снижение потерь и отсутствие явного переобучения.
- Однослойная [120] имеет самый высокий показатель точности на валидации (0.9412), но при этом тестовая точность упала до 0.7857. Это классический признак переобучения: модель с избыточной емкостью хорошо запомнила обучающие данные, но потеряла способность обобщать на новые примеры.
- Тот факт, что архитектуры со значительно меньшим числом нейронов в первом слое, чем размерность входных данных (60 признаков), показали вполне конкурентные результаты, говорит об избыточности в исходных данных. Архитектура с 15ю нейронами достигла тестовой точности 0.8095, что лишь незначительно уступает архитектуре в 60 нейронов (0.8333), имеющей в 4 раза больше нейронов. А архитектура с 30ю нейронами вообще не уступает в точности базовой модели. Таким

образом, сеть способна эффективно сжимать информацию с 60 входных измерений до гораздо меньшего числа нейронов (15–30) без критической потери качества. Данная избыточность может быть обнаружена путем исследования данных на корреляцию через тепловую карту (рисунок 5).

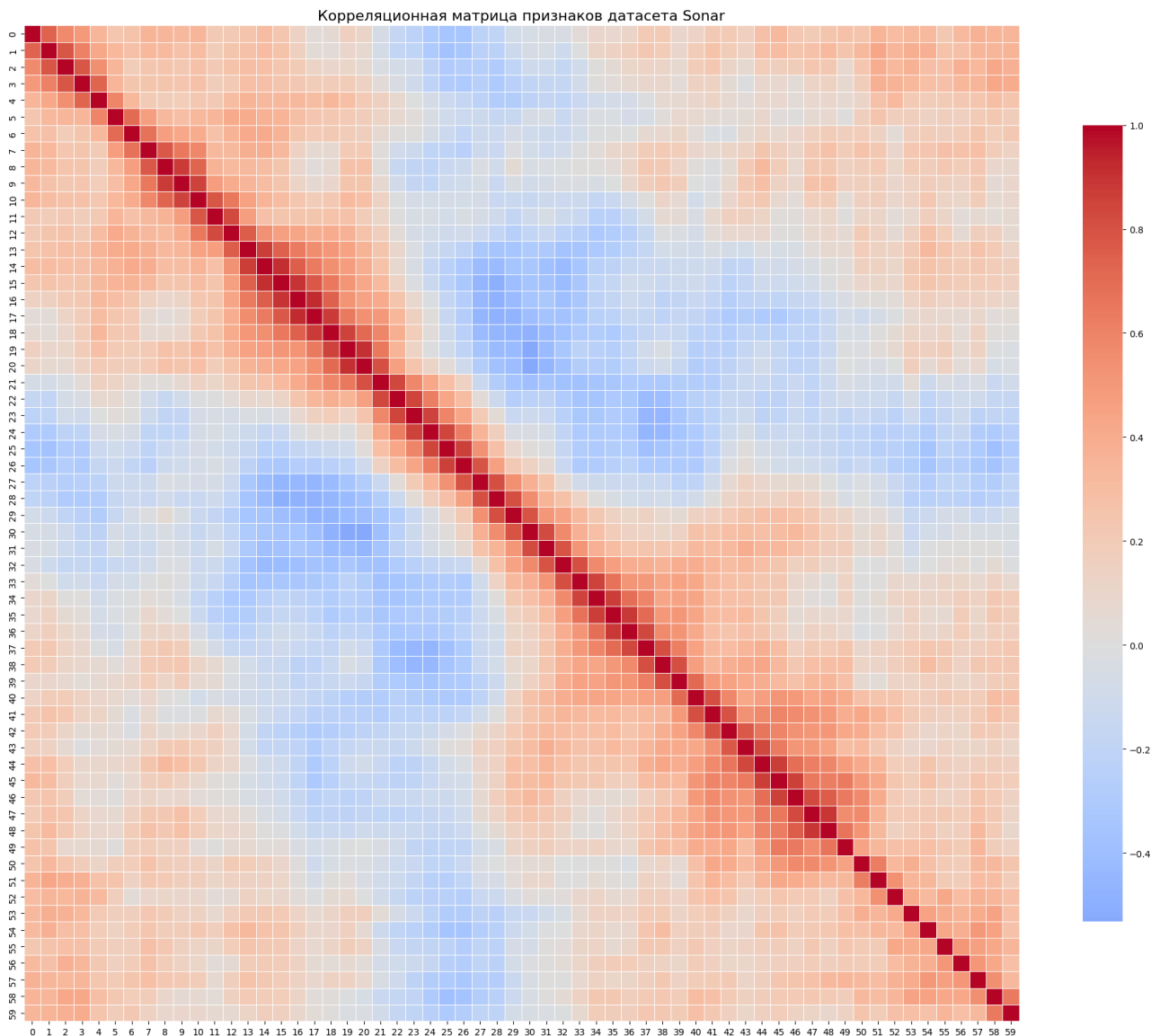


Рисунок 5. Корреляционная матрица признаков датасета.

- Двухслойные архитектуры показали разнообразные результаты. Лучший тестовый результат достигнут архитектурой [60, 30] - 0.8810. При этом точность на валидации также высока (0.9412), а кривые обучения и валидации близки друг к другу, что говорит о хорошей обобщающей способности.
- Архитектура [60, 15] также показала неплохой результат (0.8571), уступив только [60, 30]. Остальные двухслойные варианты ([30, 15] и [16, 8]) либо



не достигли высокой точности на тесте, либо сильно переобучились ([16, 8]). Для [16, 8] точность на валидации достигала 0.9412, но на тесте лишь 0.7619 – еще один пример переобучения из-за недостаточной емкости (или неудачного соотношения слоев).

- Графики потерь подтверждают эти выводы: для переобучающихся моделей ([120], [16, 8]) наблюдается расхождение кривых потерь на обучении и валидации после некоторого числа эпох. Для хорошо обобщающих моделей кривые идут близко и плавно снижаются.

Таким образом, оптимальной архитектурой для данной задачи оказалась двухслойная сеть [60, 30]. Она обеспечивает наилучший баланс между сложностью модели и обобщающей способностью. Дальнейшее усложнение (увеличение числа нейронов до 120 в однослойной сети) или неоптимальный подбор размеров слоев приводят к падению тестовой точности.

### **Выводы.**

В ходе лабораторной работы была реализована классификация сигналов сонара на два класса с помощью полносвязных нейронных сетей различной архитектуры. Проведенные эксперименты позволяют сделать следующие выводы:

- Архитектура сети существенно влияет на качество классификации. Простые однослойные модели ([15], [30], [60]) обеспечивают приемлемую, но не максимальную точность (0.81–0.83). Слишком мощная однослойная сеть ([120]) склонна к переобучению.
- Данные обладают высокой степенью избыточности. Высокие результаты, достигнутые архитектурами с существенно меньшим числом нейронов в первом слое (например, [15], [30]), чем размерность входных данных (60 признаков), указывают на сильную коррелированность исходных признаков.
- Двухслойные архитектуры могут превосходить однослойные при правильном подборе размеров. Наилучший результат (0.8810) получен

для сети [60, 30]. Это говорит о том, что добавление второго слоя позволяет модели извлекать более сложные закономерности, не теряя обобщающей способности.

- Важно контролировать переобучение. Модели с большим числом параметров ([120], [16, 8]) демонстрируют высокую точность на валидации, но низкую на тесте. Это подчеркивает необходимость использования методов регуляризации или ранней остановки при обучении сложных архитектур.
- Графики обучения являются полезным инструментом диагностики. Расхождение кривых точности/потерь на обучении и валидации служит индикатором переобучения. Стабильное снижение обеих кривых свидетельствует о хорошем обобщении.

Лучшая модель по итогам экспериментов: двухслойная сеть с архитектурой [60, 30], достигшая точности 0.8810 на тестовой выборке. Данная архитектура может быть рекомендована для дальнейшего использования на подобных данных.

## ПРИЛОЖЕНИЕ А

### ИСХОДНЫЙ КОД ПРОГРАММЫ

Название файла: lab2.ipynb

```
"""# Установка зависимостей
"""

!pip install pandas numpy matplotlib scikit-learn tensorflow

"""# Импорт"""

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.preprocessing import LabelEncoder
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score

from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
from tensorflow.keras.optimizers import Adam

"""# Загрузка данных"""

dataframe = pd.read_csv("sonar.csv", header=None)
print(dataframe.head())

"""# Предобработка данных
"""

X = dataset[:, 0:60].astype(float)
Y = dataset[:, 60]

encoder = LabelEncoder()
encoder.fit(Y)
encoded_Y = encoder.transform(Y)

X_train, X_test, y_train, y_test = train_test_split(
    X, encoded_Y, test_size=0.2, random_state=42, stratify=encoded_Y
)

"""# Исследование данных"""

X = dataframe.iloc[:, :-1]
corr_matrix = X.corr()

plt.figure(figsize=(20, 16))
sns.heatmap(corr_matrix, cmap='coolwarm', center=0,
```

```

        square=True, linewidths=0.5, cbar_kws={"shrink": 0.8})
plt.title('Корреляционная матрица признаков датасета Sonar', fontsize=16)
plt.tight_layout()
plt.show()

"""# Генерация модели

"""

def create_model(architecture, input_dim=60):
    model = Sequential()
    model.add(Dense(architecture[0], input_dim=input_dim, activation='relu'))
    for units in architecture[1:]:
        model.add(Dense(units, activation='relu'))
    model.add(Dense(1, activation='sigmoid'))
    model.compile(optimizer='adam', loss='binary_crossentropy', metrics=['accuracy'])
    return model

"""# Сравнение архитектур"""

architectures = [
    [15],      # очень маленькая однослойная
    [30],      # средняя однослойная
    [60],      # базовая однослойная
    [120],     # большая однослойная
    [60, 30],  # двухслойная: сжатие 60-30
    [60, 15],  # двухслойная: сильное сжатие 60-15
    [30, 15],  # двухслойная: уже сжато 30-15
    [16, 8]   # двухслойная: маленькая 16-8
]
architecture_results = []

for arch in architectures:
    print(f"\nОбучение архитектуры: {arch}")
    model = create_model(arch)
    history = model.fit(
        X_train, y_train,
        epochs=100,
        batch_size=10,
        validation_split=0.1,
        verbose=0
    )
    test_loss, test_acc = model.evaluate(X_test, y_test, verbose=0)
    architecture_results.append({
        'architecture': arch,
        'final_train_acc': history.history['accuracy'][-1],
        'final_val_acc': history.history['val_accuracy'][-1],
        'test_acc': test_acc,
        'history': history
    })
    print(f"\tТочность на валидации: {history.history['val_accuracy'][-1]:.4f}")
    print(f"\tТочность на тесте: {test_acc:.4f}")

```

```
"""## Графики потерь и точности"""
```

```
def plot_accuracy_grid(architecture_results, architectures):
    n = len(architectures)
    cols = 2
    rows = (n + cols - 1) // cols
    fig, axes = plt.subplots(rows, cols, figsize=(14, 5 * rows))
    fig.suptitle('Графики точности для всех архитектур', fontsize=14, y=1.02)
    axes_flat = axes.flatten() if n > 1 else [axes]

    for idx, arch in enumerate(architectures):
        result = next(r for r in architecture_results if r['architecture'] == arch)
        history = result['history']
        ax = axes_flat[idx]
        ax.plot(history.history['accuracy'], label='Обучение')
        ax.plot(history.history['val_accuracy'], label='Валидация')
        ax.set_xlabel('Эпоха', fontsize=10)
        ax.set_ylabel('Точность', fontsize=10)
        ax.set_title(f'Архитектура {arch}\nТест: {result["test_acc"]:.3f}',
                     fontsize=11, fontweight='bold')
        ax.legend(loc='lower right', fontsize=9)
        ax.grid(True, alpha=0.3)
        ax.text(0.02, 0.02, f'Val: {result["final_val_acc"]:.3f}',
                transform=ax.transAxes, fontsize=9,
                bbox=dict(boxstyle='round', facecolor='wheat', alpha=0.8))

    # Скрыть неиспользуемые подграфики
    for j in range(len(architectures), len(axes_flat)):
        axes_flat[j].axis('off')

    plt.tight_layout()
    plt.subplots_adjust(top=0.95, hspace=0.4, wspace=0.1)
    plt.show()
```

```
def plot_loss_grid(architecture_results, architectures):
    n = len(architectures)
    cols = 2
    rows = (n + cols - 1) // cols
    fig, axes = plt.subplots(rows, cols, figsize=(14, 5 * rows))
    fig.suptitle('Графики потерь для всех архитектур', fontsize=14, y=1.02)
    axes_flat = axes.flatten() if n > 1 else [axes]

    for idx, arch in enumerate(architectures):
        result = next(r for r in architecture_results if r['architecture'] == arch)
        history = result['history']
        ax = axes_flat[idx]
        ax.plot(history.history['loss'], 'b-', linewidth=2, label='Обучение')
        ax.plot(history.history['val_loss'], 'r-', linewidth=2, label='Валидация')
        ax.set_xlabel('Эпоха', fontsize=10)
        ax.set_ylabel('Потери', fontsize=10)
```

```

ax.set_title(f'Архитектура {arch}', fontsize=11, fontweight='bold')
ax.legend(loc='upper right', fontsize=9)
ax.grid(True, alpha=0.3)
ax.text(0.02, 0.95, f'Loss: {history.history["loss"][-1]:.3f}',
        transform=ax.transAxes, fontsize=9,
        bbox=dict(boxstyle='round', facecolor='lightblue', alpha=0.8))
ax.text(0.02, 0.85, f'Val Loss: {history.history["val_loss"][-1]:.3f}',
        transform=ax.transAxes, fontsize=9,
        bbox=dict(boxstyle='round', facecolor='lightcoral', alpha=0.8))

for j in range(len(architectures), len(axes_flat)):
    axes_flat[j].axis('off')

plt.tight_layout()
plt.subplots_adjust(top=0.95, hspace=0.4, wspace=0.1)
plt.show()

plot_accuracy_grid(architecture_results, architectures)
plot_loss_grid(architecture_results, architectures)

```